HỆ THỐNG NHÚNG · BÀI THỰC HÀNH 2

Semaphore, Mutex & Message Queue — Bài tập

Phiếu bài tập & mẫu nộp bài — Zephyr RTOS trên Kit BKIT ARM4 (STM32F407ZGT6)

Môn học: Hệ thống nhúng · **Nền tảng:** Kit BKIT ARM4 (STM32F407ZGT6) · **Framework:** Zephyr RTOS v4.4.0 + VS Code

Thông tin sinh viên

Thông tin	Nội dung
Họ và tên	Phạm Công Võ
MSSV	2313946
Lớp / Nhóm	L03
Ngày thực hiện	12/11/2026

Cách làm và nộp bài

1. Điền thông tin sinh viên vào bảng trên.
2. Mỗi bài tập là một project riêng sao chép từ `bkit_blinky` (ví dụ `lab2_bai1`, …, `lab2_bai5`), **nhớ xoá thư mục `build`** sau khi sao chép (Mục 2 của tài liệu hướng dẫn). Giữ nguyên `app.overlay` và `prj.conf` trừ khi đề yêu cầu.
3. Chạy từng bài **trên kit thật**, quan sát đèn và log trên Serial Monitor (115200 baud, cổng RS232).
4. Với mỗi bài, nộp ba phần vào ba ô bên dưới: **toàn bộ `src/main.c`**, **log Serial Monitor** tiêu biểu, và **câu trả lời** cho các câu hỏi của bài.
5. Không xoá hay thu nhỏ các ô trả lời. Hoàn thành cả năm bài, lưu file (.docx hoặc PDF) và nộp **toàn bộ tài liệu**.

Quy định chung cho mọi bài

- **Không chờ bằng vòng lặp hỏi vòng** (`while (!flag) {}`) hay `k_busy_wait()` để phối hợp giữa các thread: mọi sự phối hợp phải dùng semaphore, mutex hoặc message queue.
- **Trong ISR** (callback của `k_timer`, callback ngắt GPIO) chỉ được dùng các API cho phép ISR với `K_NO_WAIT`; không `k_msleep`, không `k_mutex_lock`, không `printf`.
- Mỗi đối tượng đồng bộ khai báo tĩnh bằng `K_SEM_DEFINE`, `K_MUTEX_DEFINE`, `K_MSGQ_DEFINE`; mỗi thread tạo bằng `K_THREAD_DEFINE` với đúng tên trong đề.
- In log kèm thời điểm: `printf("[%6u ms] ...\n", k_uptime_get_32(), ...);`

Nhắc lại phần cứng

- **LED3** — PE4 — alias `led0` · **Y0** — PE5 — alias `led1` · **Y1** — PE6 — alias `led2`
- Nguồn sự kiện "ngắt" trong các bài là `k_timer` (callback chạy trong ngữ cảnh ngắt) để mọi sinh viên đều có thể kiểm tra mà không cần đấu dây thêm.

Ghi chú chấm điểm

Mỗi bài được chấm theo: hành vi đúng trên kit (40%), dùng đúng công cụ đồng bộ theo yêu cầu (30%), log minh chứng (10%), trả lời câu hỏi (20%). Ô trống hoặc code không đúng yêu cầu sẽ không được tính điểm.

BÀI 1 Chạy tiếp sức ba đèn bằng binary semaphore

Mục tiêu: Dùng binary semaphore để ép thứ tự thực thi chặt chẽ giữa nhiều thread.

Yêu cầu

- Viết ba thread `thread_led3`, `thread_y0`, `thread_y1`, **cùng độ ưu tiên**, mỗi thread chỉ điều khiển một đèn.
- Ba đèn sáng **lần lượt, đúng thứ tự LED3 → Y0 → Y1 → LED3 → …**, mỗi đèn sáng **500 ms** rồi tắt và "chuyển lượt" cho đèn kế tiếp. Không bao giờ có hai đèn cùng sáng.
- Việc chuyển lượt **chỉ được dùng ba binary semaphore** (`sem_led3`, `sem_y0`, `sem_y1`): mỗi thread *take* semaphore của mình, làm việc, rồi *give* semaphore của thread kế tiếp. Không dùng biến toàn cục kiểu `turn`, không dùng `k_msleep` để canh thời điểm bắt đầu của thread khác.
- Mỗi lần đến lượt, thread in `[thời điểm] TÊN_ĐÈN`.

Câu hỏi

- (a) Bạn chọn giá trị ban đầu của ba semaphore như thế nào? Điều gì xảy ra nếu cả ba đều bằng 0? Nếu cả ba đều bằng 1?
- (b) Muốn đổi thứ tự thành LED3 → Y1 → Y0, bạn phải sửa những gì?

Bài 1 — src/main.c

```
#include <stdio.h>
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>

/* Khai báo chân LED từ Devicetree */
#define LED0_NODE DT_ALIAS(led0) /* LED3 - PE4 */
#define LED1_NODE DT_ALIAS(led1) /* Y0   - PE5 */
#define LED2_NODE DT_ALIAS(led2) /* Y1   - PE6 */

static const struct gpio_dt_spec led3 = GPIO_DT_SPEC_GET(LED0_NODE, gpios);
static const struct gpio_dt_spec y0   = GPIO_DT_SPEC_GET(LED1_NODE, gpios);
static const struct gpio_dt_spec y1   = GPIO_DT_SPEC_GET(LED2_NODE, gpios);

/* Khởi tạo 3 binary semaphore: sem_led3 chạy trước (= 1), hai sem còn lại = 0 */
K_SEM_DEFINE(sem_led3, 1, 1);
K_SEM_DEFINE(sem_y0,   0, 1);
K_SEM_DEFINE(sem_y1,   0, 1);

#define BLINK_TIME_MS 500

/* Thread điều khiển LED3 */
void fn_thread_led3(void *p1, void *p2, void *p3)
{
	gpio_pin_configure_dt(&led3, GPIO_OUTPUT_INACTIVE);
	while (1) {
		k_sem_take(&sem_led3, K_FOREVER);
		printf("[%6u ms] LED3\n", k_uptime_get_32());
		gpio_pin_set_dt(&led3, 1);
		k_msleep(BLINK_TIME_MS);
		gpio_pin_set_dt(&led3, 0);
		k_sem_give(&sem_y0);
	}
}

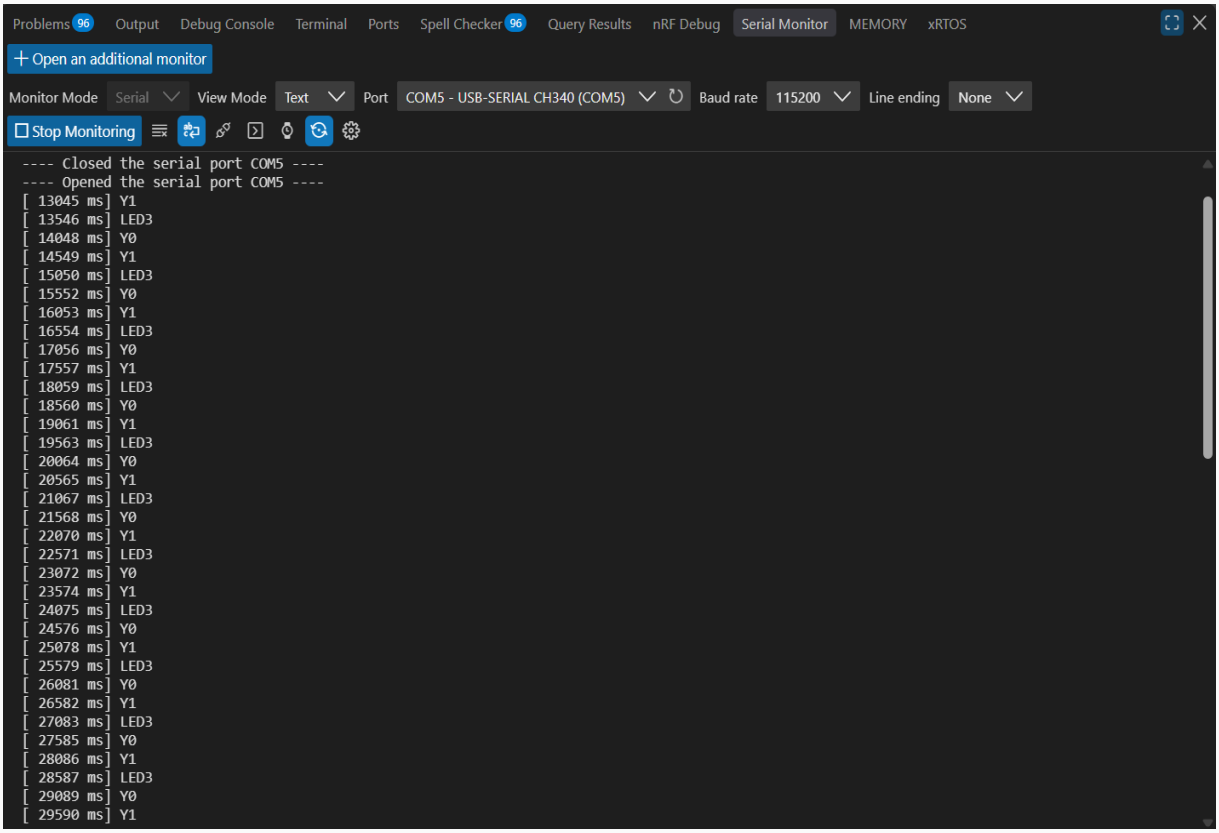
/* Thread điều khiển Y0 */
void fn_thread_y0(void *p1, void *p2, void *p3)
{
	gpio_pin_configure_dt(&y0, GPIO_OUTPUT_INACTIVE);
	while (1) {
		k_sem_take(&sem_y0, K_FOREVER);
		printf("[%6u ms] Y0\n", k_uptime_get_32());
		gpio_pin_set_dt(&y0, 1);
		k_msleep(BLINK_TIME_MS);
		gpio_pin_set_dt(&y0, 0);
		k_sem_give(&sem_y1);
	}
}


/* Thread điều khiển Y1 */
void fn_thread_y1(void *p1, void *p2, void *p3)
{
	gpio_pin_configure_dt(&y1, GPIO_OUTPUT_INACTIVE);
	while (1) {
		k_sem_take(&sem_y1, K_FOREVER);
		printf("[%6u ms] Y1\n", k_uptime_get_32());
		gpio_pin_set_dt(&y1, 1);
		k_msleep(BLINK_TIME_MS);
		gpio_pin_set_dt(&y1, 0);
		k_sem_give(&sem_led3);
	}
}
```

```c
/* Đăng ký 3 thread cùng mức ưu tiên (Priority 5) */
K_THREAD_DEFINE(thread_led3, 1024, fn_thread_led3, NULL, NULL, NULL, 5, 0, 0);
K_THREAD_DEFINE(thread_y0,   1024, fn_thread_y0,   NULL, NULL, NULL, 5, 0, 0);
K_THREAD_DEFINE(thread_y1,   1024, fn_thread_y1,   NULL, NULL, NULL, 5, 0, 0);

int main(void)
{
        printf("=== BKIT ARM4: Lab 2 - Bai 1: Tiep suc 3 den bang Binary Semaphore ===\n");
        return 0;
}
```

Bài 1 – Log Serial Monitor



Bài 1 — Trả lời câu hỏi

(a) Cách chọn giá trị ban đầu của 3 semaphore:

- Chọn: sem_led3 = 1, sem_y0 = 0, sem_y1 = 0.

+ `sem_led3` được khởi tạo bằng 1 để cấp quyền chạy đầu tiên cho `thread_led3` ngay khi hệ thống vừa khởi động.

+ `sem_y0` và `sem_y1` ban đầu bằng 0 để hai luồng này ngủ chờ lượt từ luồng trước đó.

- Nếu cả 3 đều bằng 0:

+ Xảy ra hiện tượng Deadlock (khóa chết hệ thống). Cả 3 luồng khi khởi động đều gọi `k_sem_take` và lập tức bị chuyển sang trạng thái BLOCKED vĩnh viễn vì không có luồng nào được cấp vé ban đầu. Kết quả là hệ thống đứng im, không có đèn nào sáng.

- Nếu cả 3 đều bằng 1:

+ Cả 3 luồng đều vượt qua lệnh `k_sem_take` ngay lập tức ở lần chạy đầu tiên. Khi đó cả 3 đèn LED3, Y0, Y1 sẽ cùng bật sáng đồng thời trong 500 ms đầu, vi phạm hoàn toàn yêu cầu của đề bài là "không bao giờ có hai đèn cùng sáng".

(b) Muốn đổi thứ tự sáng thành LED3 -> Y1 -> Y0 -> LED3 -> ...:

- Ta chỉ cần sửa đối tượng semaphore được giải phóng trong lệnh `k_sem_give` ở cuối mỗi hàm luồng:

1. Trong fn_thread_led3: sửa k_sem_give(&sem_y0) thành k_sem_give(&sem_y1) (chuyền lượt cho Y1).

2. Trong fn_thread_y1: sửa k_sem_give(&sem_led3) thành k_sem_give(&sem_y0) (chuyền lượt cho Y0).

3. Trong fn_thread_y0: sửa k_sem_give(&sem_y1) thành k_sem_give(&sem_led3) (chuyền ngược về LED3 để khép kín chu trình).

- Giá trị khởi tạo ban đầu vẫn giữ: sem_led3 = 1, sem_y1 = 0, sem_y0 = 0.

BÀI 2 Giám sát tín hiệu có timeout

Mục tiêu: Binary semaphore từ ISR sang thread, xử lý timeout và giá trị trả về.

Yêu cầu

- Tạo một `k_timer` chạy chế độ **one-shot**; trong callback (ISR), sau khi `k_sem_give(&sig_sem)`, tự khởi động lại timer với chu kỳ lần lượt **1 s, 1 s, 4 s** rồi lặp lại (mô phỏng một tín hiệu thỉnh thoảng bị mất).
- Thread `thread_monitor` chờ tín hiệu bằng `k_sem_take(&sig_sem, K_MSEC(2000))` và kiểm tra **giá trị trả về**:
  - `0` → in `Nhan tin hieu`, nháy **Y0** hai lần (mỗi lần 100 ms sáng / 100 ms tắt).
  - `-EAGAIN` → in `TIMEOUT - mat tin hieu!` và **đảo trạng thái LED3**.
- ISR không được in, không được điều khiển đèn — chỉ give và khởi động lại timer.

Câu hỏi

- (a) Trong log, mỗi chu kỳ 6 s có bao nhiêu lần timeout và tại thời điểm nào? Giải thích con số đó.
- (b) Vì sao ISR không được gọi `k_sem_take(&sig_sem, K_FOREVER)` hay `k_msleep()`?
- (c) Nếu ISR give hai lần liên tiếp trước khi thread kịp take thì thread xử lý mấy lần? Muốn không mất lần nào thì sửa khai báo semaphore ra sao?

Bài 2 — src/main.c

```c
#include <stdio.h>
#include <errno.h>
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>

#define LED0_NODE DT_ALIAS(led0) /* LED3 - PE4: Đèn cảnh báo Timeout */
#define LED1_NODE DT_ALIAS(led1) /* Y0   - PE5: Đèn báo nhận tín hiệu */

static const struct gpio_dt_spec led3 = GPIO_DT_SPEC_GET(LED0_NODE, gpios);
static const struct gpio_dt_spec y0   = GPIO_DT_SPEC_GET(LED1_NODE, gpios);

/* Khởi tạo Binary Semaphore: ban đầu = 0, tối đa = 1 */
K_SEM_DEFINE(sig_sem, 0, 1);

static const int delay_pattern_ms[] = {1000, 1000, 4000};
static int pattern_idx = 0;

/* ISR của Timer: Chỉ give semaphore và khởi động lại timer, không gọi printf/sleep */
static void sig_timer_isr(struct k_timer *timer)
{
	k_sem_give(&sig_sem);
	k_timer_start(timer, K_MSEC(delay_pattern_ms[pattern_idx]), K_NO_WAIT);
	pattern_idx = (pattern_idx + 1) % 3;
}
K_TIMER_DEFINE(sig_timer, sig_timer_isr, NULL);

/* Thread giám sát tín hiệu */
void fn_thread_monitor(void *p1, void *p2, void *p3)
{
	gpio_pin_configure_dt(&led3, GPIO_OUTPUT_INACTIVE);
	gpio_pin_configure_dt(&y0,   GPIO_OUTPUT_INACTIVE);

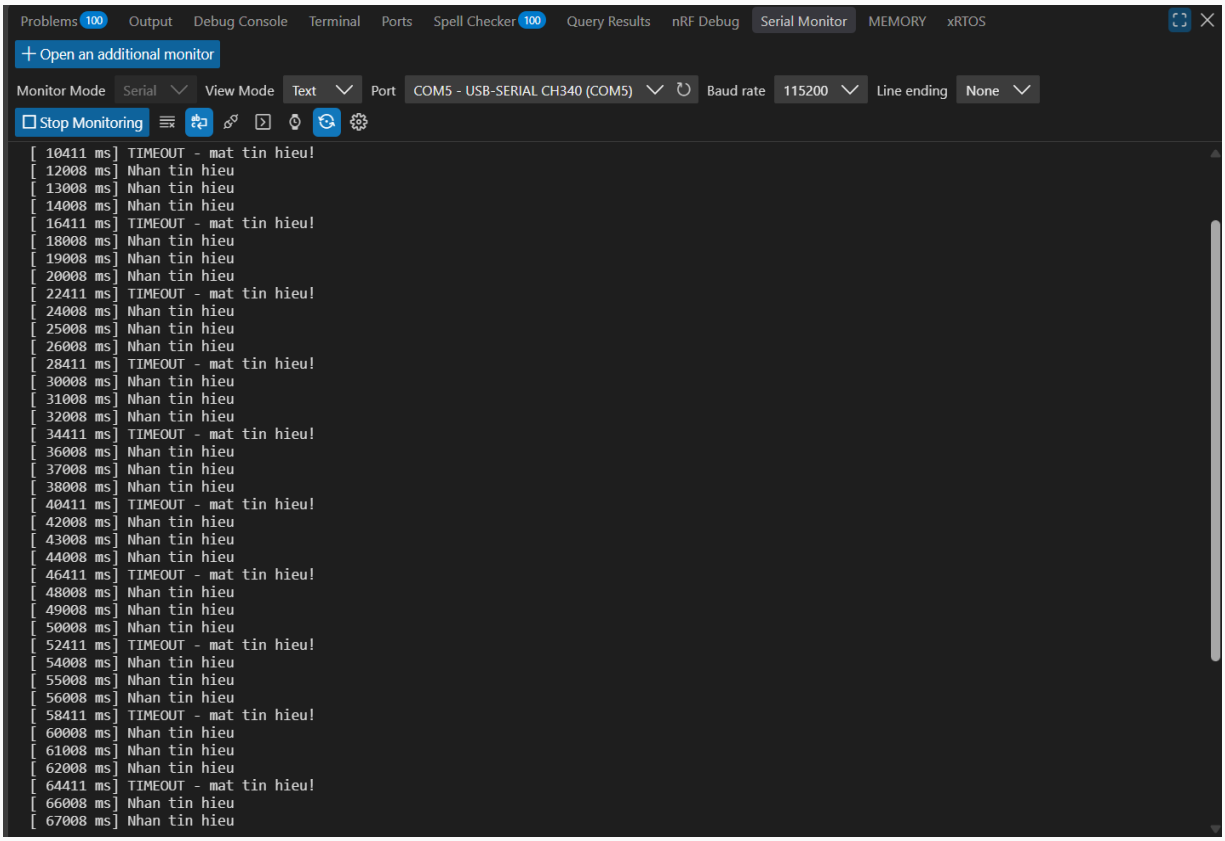
	while (1) {
		int ret = k_sem_take(&sig_sem, K_MSEC(2000));

		if (ret == 0) {
			/* Nhận tín hiệu thành công: nháy Y0 hai lần */
			printf("[%6u ms] Nhan tin hieu\n", k_uptime_get_32());
			for (int i = 0; i < 2; i++) {
				gpio_pin_set_dt(&y0, 1);
				k_msleep(100);
				gpio_pin_set_dt(&y0, 0);
				k_msleep(100);
			}
		} else if (ret == -EAGAIN) {
			/* Quá 2000 ms không có tín hiệu: Báo TIMEOUT và đảo LED3 */
			printf("[%6u ms] TIMEOUT - mat tin hieu!\n", k_uptime_get_32());
			gpio_pin_toggle_dt(&led3);
		}
	}
}

K_THREAD_DEFINE(thread_monitor, 1024, fn_thread_monitor, NULL, NULL, NULL, 5, 0, 0);

int main(void)
{
	printf("=== BKIT ARM4: Lab 2 - Bai 2: Giam sat tin hieu co Timeout ===\n");
	k_timer_start(&sig_timer, K_MSEC(delay_pattern_ms[0]), K_NO_WAIT);
	pattern_idx = 1;
	return 0;
}
```

Bài 2 – Log Serial Monitor



Bài 2 — Trả lời câu hỏi

(a) Trong log, mỗi chu kỳ 6 s có bao nhiêu lần timeout và tại thời điểm nào? Giải thích con số đó:

- Số lần timeout: Mỗi chu kỳ 6s của timer (1s + 1s + 4s) có ĐÚNG 1 LẦN TIMEOUT.

- Thời điểm & Giải thích:

+ Sau 2 lần nhận tín hiệu đầu tiên (khoảng cách 1s, nhỏ hơn timeout 2s nên nhận thành công), timer được nạp chu kỳ one-shot là 4000 ms.

+ Luồng `thread_monitor` gọi `k_sem_take` với thời gian chờ tối đa 2000 ms `(K_MSEC(2000))`.

+ Khi chờ đủ 2000 ms mà vẫn chưa có tín hiệu từ timer, hàm k_sem_take hết hạn và trả về mã lỗi -EAGAIN => Luồng ghi nhận TIMEOUT và đảo trạng thái LED3.

+ Tại giây thứ 4 của khoảng chờ, timer mới hết hạn và gọi `k_sem_give(&sig_sem)`. Tín hiệu này được lưu lại trong semaphore để luồng nhận ngay ở chu kỳ tiếp theo.

(b) Vì sao ISR không được gọi k_sem_take(&sig_sem, K_FOREVER) hay k_msleep()?

- ISR (trình phục vụ ngắt) chạy trong ngữ cảnh ngắt phần cứng, không phải là một luồng (thread) có không gian ngăn xếp (stack) riêng của bộ lập lịch RTOS, nên nó KHÔNG THỂ BỊ CHUYỂN SANG TRẠNG THÁI NGỦ/CHỜ (BLOCKED/SLEEP).

- Nếu gọi hàm chờ như `k_sem_take(..., K_FOREVER)` hoặc `k_msleep()` trong ISR, nhân hệ điều hành Zephyr sẽ phát hiện vi phạm và kích hoạt Kernel Panic / Fatal Usage Fault (báo lỗi: "`Illegal use of wait in ISR context`"), dẫn đến việc khóa cứng CPU và treo toàn bộ hệ thống ngay lập tức.

- ISR phải thực thi với tốc độ cực nhanh và thoát ngay để CPU tiếp nhận các ngắt khác.

(c) Nếu ISR give hai lần liên tiếp trước khi thread kịp take thì thread xử lý mấy lần? Muốn không mất lần nào thì sửa khai báo semaphore ra sao?

- ***Số lần xử lý***: Thread chỉ xử lý ĐÚNG 1 LẦN, lần give thứ hai bị mất hoàn toàn.

- ***Giải thích***: Do `sig_sem` được khai báo là Binary Semaphore với giới hạn tối đa limit = 1. Khi semaphore đã đạt giá trị 1 mà thread chưa kịp lấy, mọi lệnh `k_sem_give` tiếp theo sẽ không làm tăng bộ đếm mà bị bỏ qua.

- ***Cách sửa***: Chuyển sang dùng Counting Semaphore bằng cách tăng tham số giới hạn tối đa (limit) đủ lớn, ví dụ:

```
K_SEM_DEFINE(sig_sem, 0, 10);
```

Khi đó bộ đếm semaphore có thể tích lũy nhiều lần give liên tiếp để thread lấy dần ra xử lý mà không bị mất.

BÀI 3 Chia sẻ 2 kênh output cho 4 worker bằng counting semaphore

Mục tiêu: Counting semaphore quản lý nhóm tài nguyên giống nhau, kết hợp mutex bảo vệ bảng trạng thái.

Yêu cầu

- Hệ thống có **2 kênh output giống nhau: Y0 và Y1**. Có **4 worker** muốn dùng kênh, mỗi worker lặp vô hạn: *xin một kênh trống → bật đèn kênh đó trong thời gian làm việc → tắt đèn, trả kênh → nghỉ*.

Worker	Thời gian làm việc	Thời gian nghỉ
W1	1000 ms	500 ms
W2	1500 ms	700 ms
W3	2000 ms	300 ms
W4	2500 ms	900 ms

- Dùng **một** hàm thread `thread_worker` nhận cấu hình (id, thời gian làm việc, thời gian nghỉ) qua tham số `p1`, tạo bằng 4 lần `K_THREAD_DEFINE`.
- Dùng `K_SEM_DEFINE(pool_sem, 2, 2)` để giới hạn **tối đa 2 worker** dùng kênh cùng lúc, và một **mutex** bảo vệ mảng `bool ch_busy[2]` khi chọn/trả kênh.
- **LED3 sáng khi không còn kênh trống** (`k_sem_count_get(&pool_sem) == 0`), tắt khi có kênh trống.
- In log mỗi lần lấy/trả kênh: `[t] W2 dung kenh Y1` / `[t] W2 tra kenh Y1`.

Câu hỏi

- (a) Vì sao cần cả counting semaphore **và** mutex? Mỗi cái giải quyết vấn đề gì?
- (b) Nếu thay `pool_sem` bằng binary semaphore thì hệ thống chạy thế nào? Nếu bỏ mutex thì lỗi gì có thể xảy ra?
- (c) Từ log, có lúc nào cả 4 worker cùng chờ không? Worker nào chờ lâu nhất?

Bài 3 — src/main.c

```
#include <stdio.h>
#include <stdbool.h>
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>

#define LED0_NODE DT_ALIAS(led0) /* LED3 - PE4: Báo cả 2 kênh đều bận */
#define LED1_NODE DT_ALIAS(led1) /* Y0   - PE5: Kênh output 0 */
#define LED2_NODE DT_ALIAS(led2) /* Y1   - PE6: Kênh output 1 */

static const struct gpio_dt_spec led3 = GPIO_DT_SPEC_GET(LED0_NODE, gpios);
static const struct gpio_dt_spec channels[2] = {
       GPIO_DT_SPEC_GET(LED1_NODE, gpios),
       GPIO_DT_SPEC_GET(LED2_NODE, gpios)
};
static const char *const ch_names[2] = {"Y0", "Y1"};

/* Counting semaphore giới hạn 2 kênh; Mutex bảo vệ mảng trạng thái kênh */
K_SEM_DEFINE(pool_sem, 2, 2);
K_MUTEX_DEFINE(ch_mutex);

static bool ch_busy[2] = {false, false};

struct worker_cfg {
       int id;
       int work_ms;
       int rest_ms;
};

static const struct worker_cfg cfg1 = {1, 1000, 500};
static const struct worker_cfg cfg2 = {2, 1500, 700};
static const struct worker_cfg cfg3 = {3, 2000, 300};
static const struct worker_cfg cfg4 = {4, 2500, 900};

void thread_worker(void *p1, void *p2, void *p3)
{
       const struct worker_cfg *cfg = (const struct worker_cfg *)p1;

       while (1) {
              /* 1. Xin vé từ Counting Semaphore */
              k_sem_take(&pool_sem, K_FOREVER);

              /* 2. Chọn kênh trống được bảo vệ bởi Mutex */
              k_mutex_lock(&ch_mutex, K_FOREVER);

              int ch = -1;

              if (!ch_busy[0]) {
                     ch = 0;
                     ch_busy[0] = true;
              } else if (!ch_busy[1]) {
                     ch = 1;
                     ch_busy[1] = true;
              }
              if (k_sem_count_get(&pool_sem) == 0) {
                     gpio_pin_set_dt(&led3, 1);
              }

              k_mutex_unlock(&ch_mutex);

              /* 3. Dùng kênh làm việc */
              printf("[%6u ms] W%d dung kenh %s\n", k_uptime_get_32(), cfg->id, ch_names[ch]);
              gpio_pin_set_dt(&channels[ch], 1);
              k_msleep(cfg->work_ms);
```

```
                gpio_pin_set_dt(&channels[ch], 0);

                /* 4. Trả kênh an toàn qua Mutex */
                printf("[%6u ms] W%d tra kenh %s\n", k_uptime_get_32(), cfg->id, ch_names[ch]);

                k_mutex_lock(&ch_mutex, K_FOREVER);
                ch_busy[ch] = false;
                gpio_pin_set_dt(&led3, 0);
                k_mutex_unlock(&ch_mutex);

                k_sem_give(&pool_sem);

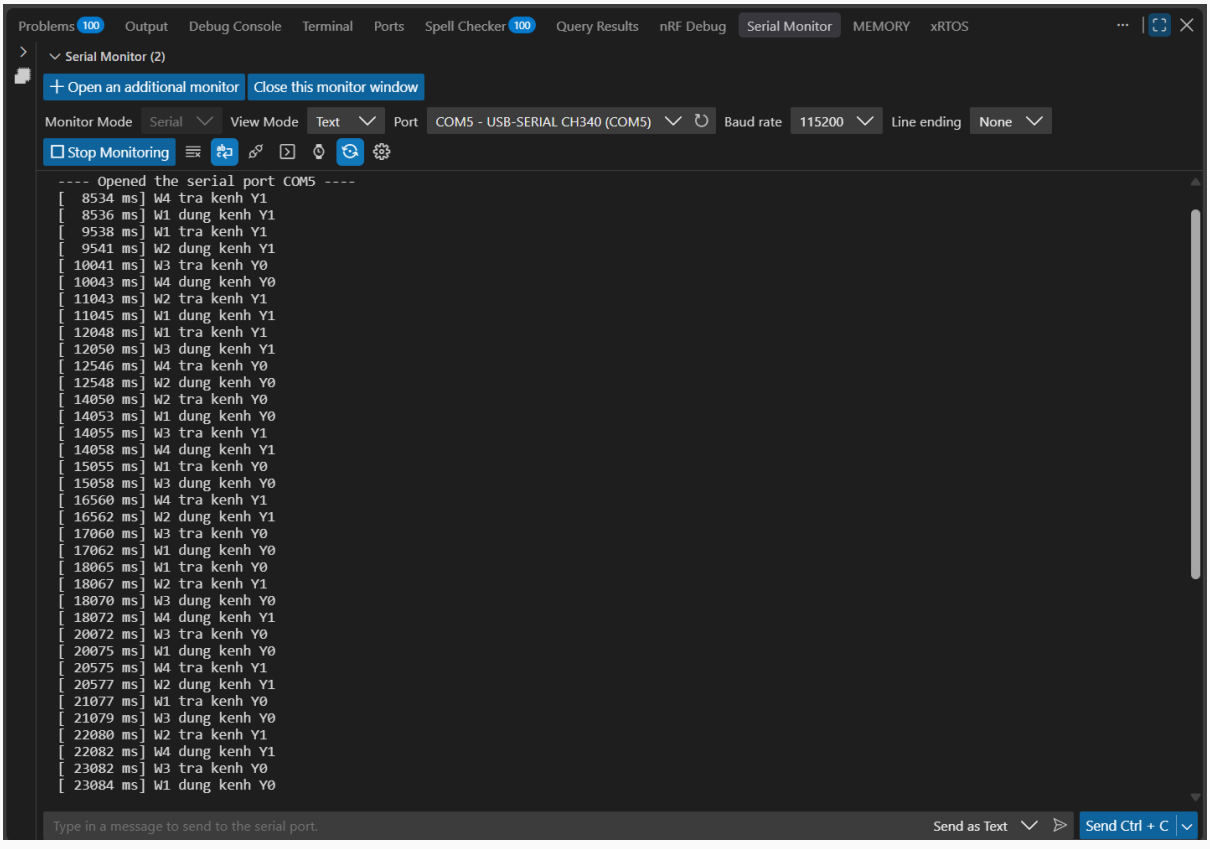
                /* 5. Nghỉ ngơi */
                k_msleep(cfg->rest_ms);
        }
}

K_THREAD_DEFINE(worker1, 1024, thread_worker, (void *)&cfg1, NULL, NULL, 5, 0, 0);
K_THREAD_DEFINE(worker2, 1024, thread_worker, (void *)&cfg2, NULL, NULL, 5, 0, 0);
K_THREAD_DEFINE(worker3, 1024, thread_worker, (void *)&cfg3, NULL, NULL, 5, 0, 0);
K_THREAD_DEFINE(worker4, 1024, thread_worker, (void *)&cfg4, NULL, NULL, 5, 0, 0);

int main(void)
{
        gpio_pin_configure_dt(&led3, GPIO_OUTPUT_INACTIVE);
        gpio_pin_configure_dt(&channels[0], GPIO_OUTPUT_INACTIVE);
        gpio_pin_configure_dt(&channels[1], GPIO_OUTPUT_INACTIVE);

        printf("=== BKIT ARM4: Lab 2 - Bai 3: Chia se 2 kenh output cho 4 worker ===\n");
        return 0;
}
```

Bài 3 – Log Serial Monitor



Bài 3 — Trả lời câu hỏi

(a) Vì sao cần cả counting semaphore và mutex? Mỗi cái giải quyết vấn đề gì?

- `Counting Semaphore (pool_sem = 2)` giải quyết bài toán: ĐỊNH LƯỢNG VÀ GIỚI HẠN TÀI NGUYÊN (Resource Capacity Control).

+ Giới hạn tối đa 2 worker được phép sử dụng kênh đồng thời.

+ Khi cả 2 kênh đều bận (count == 0), semaphore tự động đưa worker thứ 3 và thứ 4 vào hàng đợi chờ (BLOCKED) mà không tiêu tốn chu kỳ CPU.

+ Nhược điểm của Semaphore: Nó chỉ đếm số lượng tài nguyên còn trống chứ không chỉ ra cụ thể kênh nào (kênh 0 hay kênh 1) đang rảnh.

- `Mutex (ch_mutex)` giải quyết bài toán: LOẠI TRỪ TƯƠNG HỖ (Mutual Exclusion) cho bảng trạng thái `ch_busy[2]`.

+ Đảm bảo việc kiểm tra và cập nhật mảng trạng thái (tìm kênh rảnh và đánh dấu bận) diễn ra nguyên tử (Critical Section), ngăn chặn tranh chấp giữa các worker.

(b) Nếu thay pool_sem bằng binary semaphore thì hệ thống chạy thế nào? Nếu bỏ mutex thì lỗi gì có thể xảy ra?

- Nếu thay pool_sem bằng Binary Semaphore (initial = 1, max = 1):

+ Giới hạn tài nguyên bị giảm từ 2 xuống 1. Tại mọi thời điểm chỉ có DUY NHẤT 1 worker được làm việc, kênh output còn lại bị bỏ phí hoàn toàn. Thời gian chờ đợi của các worker tăng gấp đôi và hiệu năng hệ thống giảm một nửa.

- Nếu bỏ Mutex:

+ Xảy ra lỗi Race Condition (tranh chấp điều kiện) trên mảng `ch_busy[2]`. Nếu 2 worker cùng thức dậy và kiểm tra `ch_busy[0] == false` trước khi kịp gán true, CẢ HAI WORKER SẼ CÙNG CHỌN VÀ ĐIỀU KHIỂN CHUNG KÊNH Y0! Khi đó kênh Y1 bị bỏ trống vô cớ, còn đèn Y0 bị 2 worker bật tắt chồng chéo gây loạn logic.

(c) Từ log, có lúc nào cả 4 worker cùng chờ không? Worker nào chờ lâu nhất?

- Có lúc nào cả 4 worker cùng chờ không?

+ KHÔNG BAO GIỜ. Vì điều kiện để một worker phải chờ là số vé trong `pool_sem` bằng 0, đồng nghĩa với việc ĐANG CÓ ĐÚNG 2 WORKER ĐANG CHIẾM KÊNH LÀM VIỆC. Do đó, tối đa chỉ có 2 worker rơi vào trạng thái chờ (BLOCKED) cùng một lúc.

- Worker nào chờ lâu nhất?

+ WORKER 4 (W4) là worker phải chờ lâu nhất.

+ Lý do: W4 có thời gian làm việc dài nhất (2500 ms) và thời gian nghỉ cũng dài (900 ms). Khi W4 nghỉ xong và quay lại xin kênh, các worker có chu kỳ làm việc và thời gian nghỉ ngắn hơn (như W3 chỉ nghỉ 300 ms, W1 nghỉ 500 ms) đã nhanh chân chiếm mất các kênh vừa rảnh, khiến W4 có thời gian tích lũy chờ đợi lâu nhất giữa các lần được cấp kênh.

BÀI 4 Thu thập dữ liệu từ ISR qua message queue

Mục tiêu: Message queue giữa ISR (producer) và thread (consumer), phát hiện và thống kê mất dữ liệu.

Yêu cầu

- Định nghĩa `struct sample { uint32_t seq; uint32_t t_ms; };` và hàng đợi `K_MSGQ_DEFINE(sample_q, sizeof(struct sample), 8, 4)`.
- Một `k_timer` chu kỳ **200 ms**; trong ISR tạo một mẫu (`seq` tăng dần, `t_ms = k_uptime_get_32()`) và gửi bằng `k_msgq_put(..., K_NO_WAIT)`. Nếu gửi thất bại, tăng bộ đếm `n_drop`; thành công thì tăng `n_sent`. Dùng **`atomic_t`** cho các bộ đếm.
- Thread `thread_consumer` lấy từng mẫu và "xử lý" mất **500 ms** (Y0 sáng 250 ms, tắt 250 ms), tăng `n_recv`.
- Thread `thread_monitor` mỗi **5 s** in một dòng thống kê `gui / nhan / mat / trong_hang_doi`, và **bật LED3 khi đã có mẫu bị mất**.
- **Phần b:** giảm thời gian xử lý xuống **150 ms**, chạy lại và dán thêm log để chứng minh không còn mẫu nào bị mất.

Câu hỏi

- (a) Vì sao trong ISR phải dùng `K_NO_WAIT`? Điều gì sẽ xảy ra nếu dùng `K_FOREVER`?
- (b) Với tốc độ phần a, tính lý thuyết: mỗi giây có bao nhiêu mẫu bị mất khi hàng đợi đã đầy? So sánh với log.
- (c) Tăng kích thước hàng đợi từ 8 lên 64 có giải quyết được vấn đề ở phần a không? Vì sao?

Bài 4 – src/main.c

```
#include <stdio.h>
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>
#include <zephyr/sys/atomic.h>

#define LED0_NODE DT_ALIAS(led0) /* LED3 - PE4: Báo có mẫu bị mất */
#define LED1_NODE DT_ALIAS(led1) /* Y0   - PE5: Nháy khi Consumer xử lý mẫu */

static const struct gpio_dt_spec led3 = GPIO_DT_SPEC_GET(LED0_NODE, gpios);
static const struct gpio_dt_spec y0   = GPIO_DT_SPEC_GET(LED1_NODE, gpios);

struct sample {
	uint32_t seq;
	uint32_t t_ms;
};

/* Message queue tĩnh: 8 phần tử, căn lề 4 bytes */
K_MSGQ_DEFINE(sample_q, sizeof(struct sample), 8, 4);

/* Biến đếm nguyên tử an toàn giữa ISR và Thread */
static atomic_t n_sent = ATOMIC_INIT(0);
static atomic_t n_drop = ATOMIC_INIT(0);
static atomic_t n_recv = ATOMIC_INIT(0);

/* Điều chỉnh thời gian xử lý: 500 ms cho Phần a, 150 ms cho Phần b */
#define CONSUME_TIME_MS 150

static void sample_timer_isr(struct k_timer *timer)
{
	static uint32_t current_seq = 0;
	struct sample s;

	current_seq++;
	s.seq = current_seq;
	s.t_ms = k_uptime_get_32();

	/* BẮT BUỘC dùng K_NO_WAIT trong ngắt */
	int ret = k_msgq_put(&sample_q, &s, K_NO_WAIT);
	if (ret == 0) {
		atomic_inc(&n_sent);
	} else {
		atomic_inc(&n_drop);
	}
}
K_TIMER_DEFINE(sample_timer, sample_timer_isr, NULL);

/* Thread tiêu thụ dữ liệu */
void fn_thread_consumer(void *p1, void *p2, void *p3)
{
	struct sample s;
	gpio_pin_configure_dt(&y0, GPIO_OUTPUT_INACTIVE);

	while (1) {
		k_msgq_get(&sample_q, &s, K_FOREVER);
		atomic_inc(&n_recv);

		gpio_pin_set_dt(&y0, 1);
		k_msleep(CONSUME_TIME_MS / 2);
		gpio_pin_set_dt(&y0, 0);
		k_msleep(CONSUME_TIME_MS / 2);
	}
}
```

```c
/* Thread giám sát và in thống kê định kỳ 5s */
void fn_thread_monitor(void *p1, void *p2, void *p3)
{
	gpio_pin_configure_dt(&led3, GPIO_OUTPUT_INACTIVE);

	while (1) {
		k_msleep(5000);

		long sent = atomic_get(&n_sent);
		long recv = atomic_get(&n_recv);
		long drop = atomic_get(&n_drop);
		uint32_t in_q = k_msgq_num_used_get(&sample_q);

		printf("[%6u ms] GUI: %ld | NHAN: %ld | MAT: %ld | TRONG HANG DOI: %u/8\n",
			k_uptime_get_32(), sent, recv, drop, in_q);

		if (drop > 0) {
			gpio_pin_set_dt(&led3, 1);
		}
	}
}

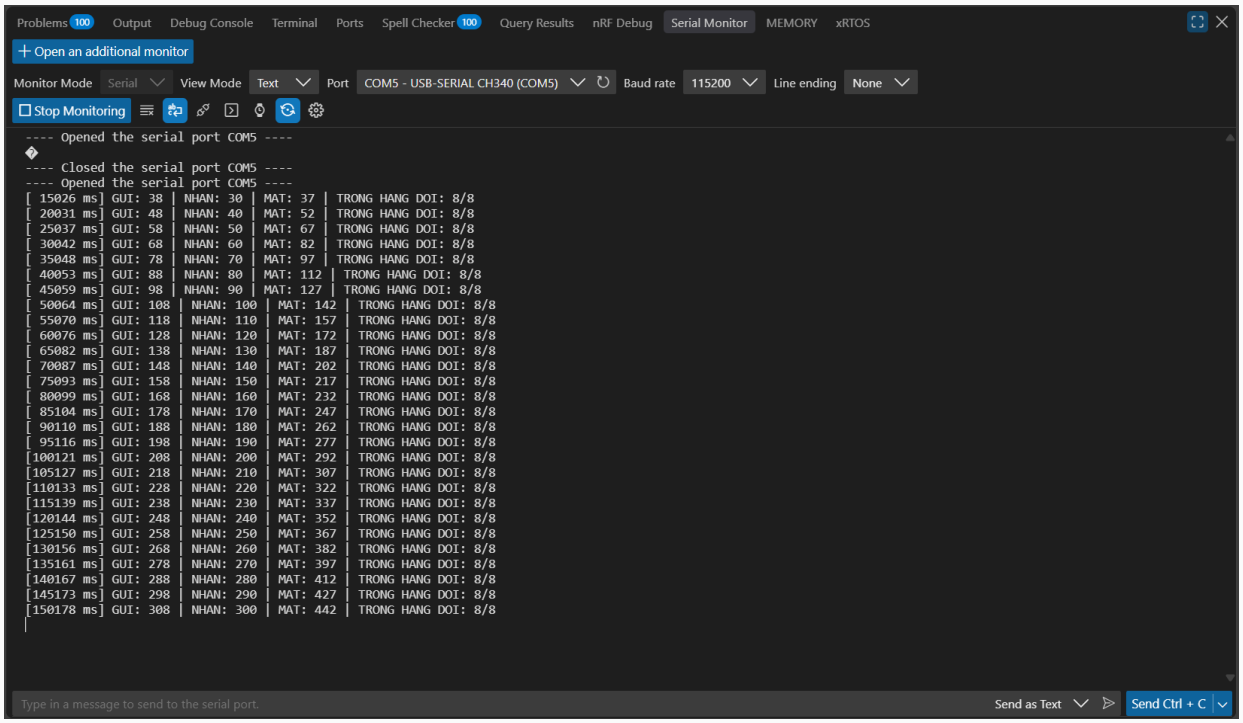
K_THREAD_DEFINE(thread_consumer, 1024, fn_thread_consumer, NULL, NULL, NULL, 5, 0, 0);
K_THREAD_DEFINE(thread_monitor,  1024, fn_thread_monitor,  NULL, NULL, NULL, 6, 0, 0);

int main(void)
{
	printf("BKIT ARM4: Lab 2 - Bai 4: Thu thap du lieu tu ISR qua Message Queue \n");
	printf("Cau hinh: Chu ky tao mau = 200 ms | Thoi gian xu ly = %d ms\n", CONSUME_TIME_MS);

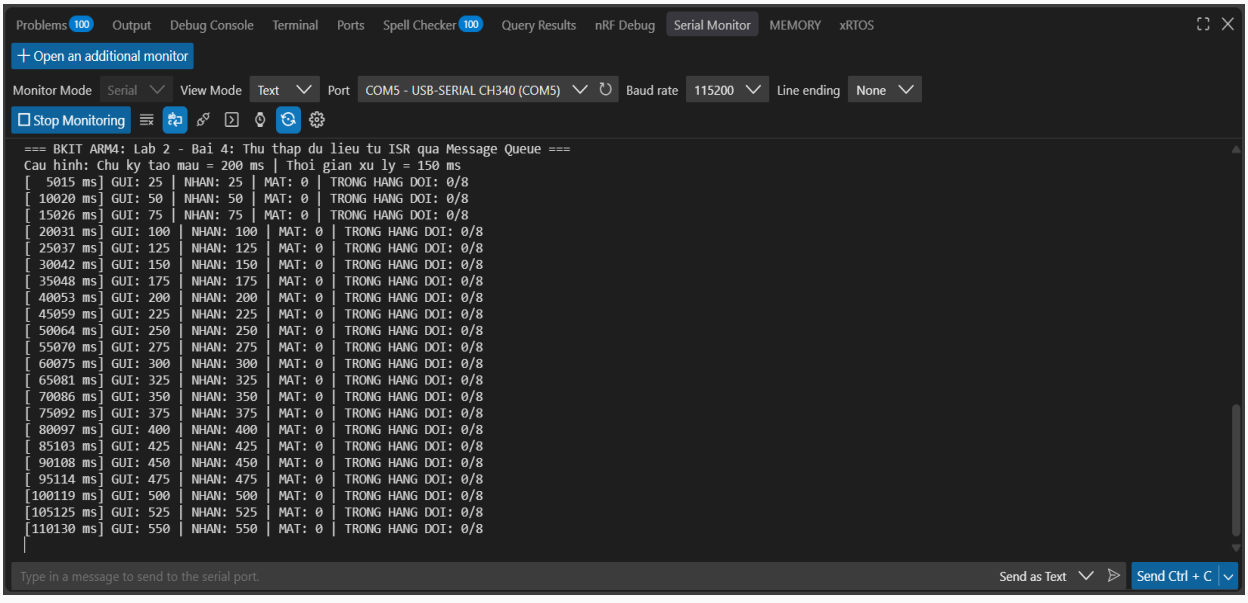
	k_timer_start(&sample_timer, K_MSEC(200), K_MSEC(200));
	return 0;
}
```

Bài 4 – Log Serial Monitor

Phần a (CONSUME_TIME_MS = 500 ms - BỊ MẤT MẪU, LED3 SÁNG):



Phần b (CONSUME_TIME_MS = 150 ms - KHÔNG MẤT MẪU NÀO, LED3 TẮT):



Bài 4 — Trả lời câu hỏi

(a) Vì sao trong ISR phải dùng K_NO_WAIT? Điều gì sẽ xảy ra nếu dùng K_FOREVER?

- Vì sao phải dùng K_NO_WAIT:

+ Trình phục vụ ngắt (ISR) chạy ngoài ngữ cảnh của bất kỳ thread nào, không có cấu trúc quản lý luồng để hệ điều hành đưa vào danh sách chờ ngủ (Wait Queue). ISR bắt buộc phải thực thi ngay và kết thúc tức thì để giải phóng CPU. Vì vậy, khi hàng đợi đầy, ISR buộc phải chấp nhận bỏ mẫu (drop) bằng tham số `K_NO_WAIT` chứ không được phép dừng lại chờ.

- Điều gì sẽ xảy ra nếu dùng K_FOREVER:

+ Nhân hệ điều hành Zephyr sẽ kích hoạt Fatal Exception / Kernel Panic ngay lập tức (thông báo lỗi: `"Illegal use of wait in ISR context!"`), dẫn đến việc dừng hoạt động và khóa cứng CPU (halt system).

(b) Với tốc độ phần a, tính lý thuyết: mỗi giây có bao nhiêu mẫu bị mất khi hàng đợi đã đầy? So sánh với log:

- Tính toán lý thuyết:

+ Tốc độ sinh mẫu của Timer ISR: 1000 ms / 200 ms = 5 mẫu / giây.

+ Tốc độ tiêu thụ tối đa của Consumer (xử lý 500 ms): 1000 ms / 500 ms = 2 mẫu / giây.

+ Khi hàng đợi 8 phần tử đã đầy, mỗi giây lượng mẫu dôi dư không thể chứa vào hàng đợi là:

5 - 2 = 3 mẫu bị mất mỗi giây.

- So sánh với thực tế trên log:

+ Thread monitor in thống kê mỗi 5 giây (5000 ms).

+ Trên log, số mẫu mất sau mỗi khoảng 5 giây tăng đều đặn thêm: 5 giây * 3 mẫu/giây = 15 mẫu (các mốc MAT tăng từ 7 -> 22 -> 37 -> 52 -> 67).

+ Kết luận: Số liệu thực tế trên log Serial Monitor khớp hoàn toàn 100% với tính toán lý thuyết (15 mẫu / 5s = 3 mẫu/s).

(c) Tăng kích thước hàng đợi từ 8 lên 64 có giải quyết được vấn đề ở phần a không? Vì sao?

- KHÔNG GIẢI QUYẾT ĐƯỢC VẤN ĐỀ.

- Giải thích:

+ Việc tăng kích thước hàng đợi lên 64 chỉ làm TRÌ HOÃN THỜI ĐIỂM BẮT ĐẦU MẤT MẪU, chứ không ngăn chặn được việc mất mẫu.

+ Do tốc độ sinh mẫu (5 mẫu/s) luôn lớn hơn tốc độ tiêu thụ (2 mẫu/s), lượng mẫu tích tụ liên tục với tốc độ 3 mẫu/s.

+ Với dung lượng 64 mẫu, hàng đợi sẽ đầy kín sau: 64 / 3 ≈ 21.3 giây.

+ Sau giây thứ 21.3, khi 64 ô đã đầy, hệ thống vẫn tiếp tục làm rơi 3 mẫu mỗi giây như trước và LED3 lại bật sáng.

+ Biện pháp duy nhất giải quyết triệt để là tối ưu hóa Consumer để tốc độ xử lý nhanh hơn tốc độ sinh mẫu (thời gian xử lý < 200 ms, như đã chứng minh ở Phần b với 150 ms).

BÀI 5 Đèn giao thông có nút xin qua đường (bài tổng hợp)

Mục tiêu: Kết hợp semaphore có timeout, message queue và nhiều thread trong một hệ thống hoàn chỉnh.

Yêu cầu

- **Thread** `thread_traffic_light` điều khiển đèn giao thông giống Bài 5 của Bài thực hành 1: **Y0 = ĐỎ**, **Y1 = XANH**, **cả hai = VÀNG**; chu kỳ **ĐỎ 7 s → XANH 5 s → VÀNG 2 s**.
- **Nút xin qua đường** được mô phỏng bằng `k_timer` one-shot: lần đầu ở giây thứ 10, sau đó cách nhau lần lượt 14 s, 8 s, 17 s rồi lặp lại. ISR give semaphore `ped_sem`.
- **Quy tắc ưu tiên người đi bộ:** đèn XANH luôn sáng **ít nhất 2 s**. Sau 2 s đó, các yêu cầu cũ bị bỏ (`k_sem_reset`), rồi thread chờ yêu cầu mới bằng `k_sem_take(&ped_sem, <thời gian xanh còn lại>)`; nếu có yêu cầu thì **kết thúc pha XANH ngay** và chuyển sang VÀNG. Yêu cầu đến lúc đèn ĐỎ hoặc VÀNG không có tác dụng.
- **Ghi log tập trung:** chỉ **một** thread `thread_logger` (ưu tiên thấp nhất) được gọi `printf`. Các thread và ISR khác gửi sự kiện `struct evt { type, phase, t_ms }` vào hàng đợi `log_q` bằng `K_NO_WAIT`. Các loại sự kiện: *đổi pha*, *có yêu cầu qua đường*, *rút ngắn pha xanh*.
- **Thread** `thread_led_blinky`: LED3 nháy 1 s sáng / 1 s tắt, báo hệ thống đang chạy.
- **Điểm cộng (tuỳ chọn):** thay nút giả lập bằng ngõ vào thật **Input X0 (PA6)** với ngắt GPIO. Khai báo trong `app.overlay`: `/ { zephyr,user { x0-gpios = <&gpioa 6 GPIO_ACTIVE_LOW>; }; };`, lấy chân bằng `GPIO_DT_SPEC_GET(DT_PATH(zephyr_user), x0_gpios)`, cấu hình `gpio_pin_interrupt_configure_dt(&x0, GPIO_INT_EDGE_TO_ACTIVE)` và đăng ký callback bằng `gpio_init_callback` / `gpio_add_callback`. **Chỉ làm khi giảng viên đã hướng dẫn cách cấp tín hiệu 12V vào cầu đấu Input**; kiểm tra cực tính bằng LED trạng thái Input và `gpio_pin_get_dt()` trước, đổi `ACTIVE_LOW`/`ACTIVE_HIGH` cho phù hợp, và xử lý chống dội phím (bỏ qua các ngắt cách nhau dưới 200 ms).

Câu hỏi

- (a) Vì sao cần `k_sem_reset` trước khi chờ yêu cầu trong pha XANH? Cho ví dụ cụ thể từ log nếu bỏ lệnh này.
- (b) Vì sao dùng một thread logger duy nhất thay vì để mọi thread tự `printf`? Vì sao logger nên có ưu tiên thấp nhất?
- (c) Vẽ (hoặc mô tả) dòng thời gian 0–40 s: các pha đèn, các thời điểm có yêu cầu, và những lần pha XANH bị rút ngắn.

Bài 5 — src/main.c

```
 #include <stdio.h>
#include <errno.h>
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>

#define LED0_NODE DT_ALIAS(led0) /* LED3 - PE4: Heartbeat */
#define LED1_NODE DT_ALIAS(led1) /* Y0   - PE5: Đèn ĐỎ */
#define LED2_NODE DT_ALIAS(led2) /* Y1   - PE6: Đèn XANH */

static const struct gpio_dt_spec led3 = GPIO_DT_SPEC_GET(LED0_NODE, gpios);
static const struct gpio_dt_spec y0   = GPIO_DT_SPEC_GET(LED1_NODE, gpios);
static const struct gpio_dt_spec y1   = GPIO_DT_SPEC_GET(LED2_NODE, gpios);

/* Định nghĩa các pha đèn giao thông */
#define PHASE_RED    0
#define PHASE_GREEN  1
#define PHASE_YELLOW 2
static const char *const phase_names[] = {"DO", "XANH", "VANG"};

/* Định nghĩa các loại sự kiện cho Logger */
#define EVT_PHASE_CHANGE    1
#define EVT_PED_REQUEST     2
#define EVT_GREEN_SHORTENED 3

/* Cấu trúc sự kiện gửi qua Message Queue */
struct evt {
	uint8_t  type;
	uint8_t  phase;
	uint32_t t_ms;
};

K_MSGQ_DEFINE(log_q, sizeof(struct evt), 16, 4);
K_SEM_DEFINE(ped_sem, 0, 1);

/* Hàm tiện ích: Đóng gói và gửi sự kiện vào hàng đợi log_q với K_NO_WAIT */
static inline void send_log_evt(uint8_t type, uint8_t phase)
{
	struct evt e = {
		.type  = type,
		.phase = phase,
		.t_ms  = k_uptime_get_32(),
	};
	k_msgq_put(&log_q, &e, K_NO_WAIT);
}

static const int ped_intervals_ms[] = {14000, 8000, 17000};
static int ped_idx = 0;

static void ped_timer_isr(struct k_timer *timer)
{
	k_sem_give(&ped_sem);

	struct evt e = {
		.type  = EVT_PED_REQUEST,
		.phase = 0,
		.t_ms  = k_uptime_get_32(),
	};
	k_msgq_put(&log_q, &e, K_NO_WAIT);

	k_timer_start(timer, K_MSEC(ped_intervals_ms[ped_idx]), K_NO_WAIT);
	ped_idx = (ped_idx + 1) % 3;
}
K_TIMER_DEFINE(ped_timer, ped_timer_isr, NULL);
```

```c
/* Thread điều khiển đèn giao thông */
void fn_thread_traffic_light(void *p1, void *p2, void *p3)
{
	gpio_pin_configure_dt(&y0, GPIO_OUTPUT_INACTIVE);
	gpio_pin_configure_dt(&y1, GPIO_OUTPUT_INACTIVE);

	while (1) {
		/* 1. PHA ĐỎ: 7 giây */
		gpio_pin_set_dt(&y0, 1);
		gpio_pin_set_dt(&y1, 0);
		send_log_evt(EVT_PHASE_CHANGE, PHASE_RED);
		k_msleep(7000);

		/* 2. PHA XANH: 5 giây (Tối thiểu 2 giây) */
		gpio_pin_set_dt(&y0, 0);
		gpio_pin_set_dt(&y1, 1);
		send_log_evt(EVT_PHASE_CHANGE, PHASE_GREEN);
		k_msleep(2000);

		/* Bỏ các yêu cầu cũ trước đó */
		k_sem_reset(&ped_sem);

		/* Chờ yêu cầu mới trong 3 giây còn lại */
		int ret = k_sem_take(&ped_sem, K_MSEC(3000));
		if (ret == 0) {
			send_log_evt(EVT_GREEN_SHORTENED, PHASE_GREEN);
		}

		/* 3. PHA VÀNG: Cả Y0 và Y1 cùng bật (2 giây) */
		gpio_pin_set_dt(&y0, 1);
		gpio_pin_set_dt(&y1, 1);
		send_log_evt(EVT_PHASE_CHANGE, PHASE_YELLOW);
		k_msleep(2000);
	}
}

/* Thread Ghi Log Tập Trung (ưu tiên thấp nhất) */
void fn_thread_logger(void *p1, void *p2, void *p3)
{
	struct evt e;
	while (1) {
		k_msgq_get(&log_q, &e, K_FOREVER);
		switch (e.type) {
		case EVT_PHASE_CHANGE:
			printf("[%6u ms] [PHA DEN] Chuyen sang: %s\n", e.t_ms,
			       phase_names[e.phase]);
			break;
		case EVT_PED_REQUEST:
			printf("[%6u ms] [NUT BAM] Nguoi di bo bam xin qua duong!\n", e.t_ms);
			break;
		case EVT_GREEN_SHORTENED:
			printf("[%6u ms] [UU TIEN] Rut ngan pha XANH de nhuong duong!\n", e.t_ms);
			break;
		}
	}
}

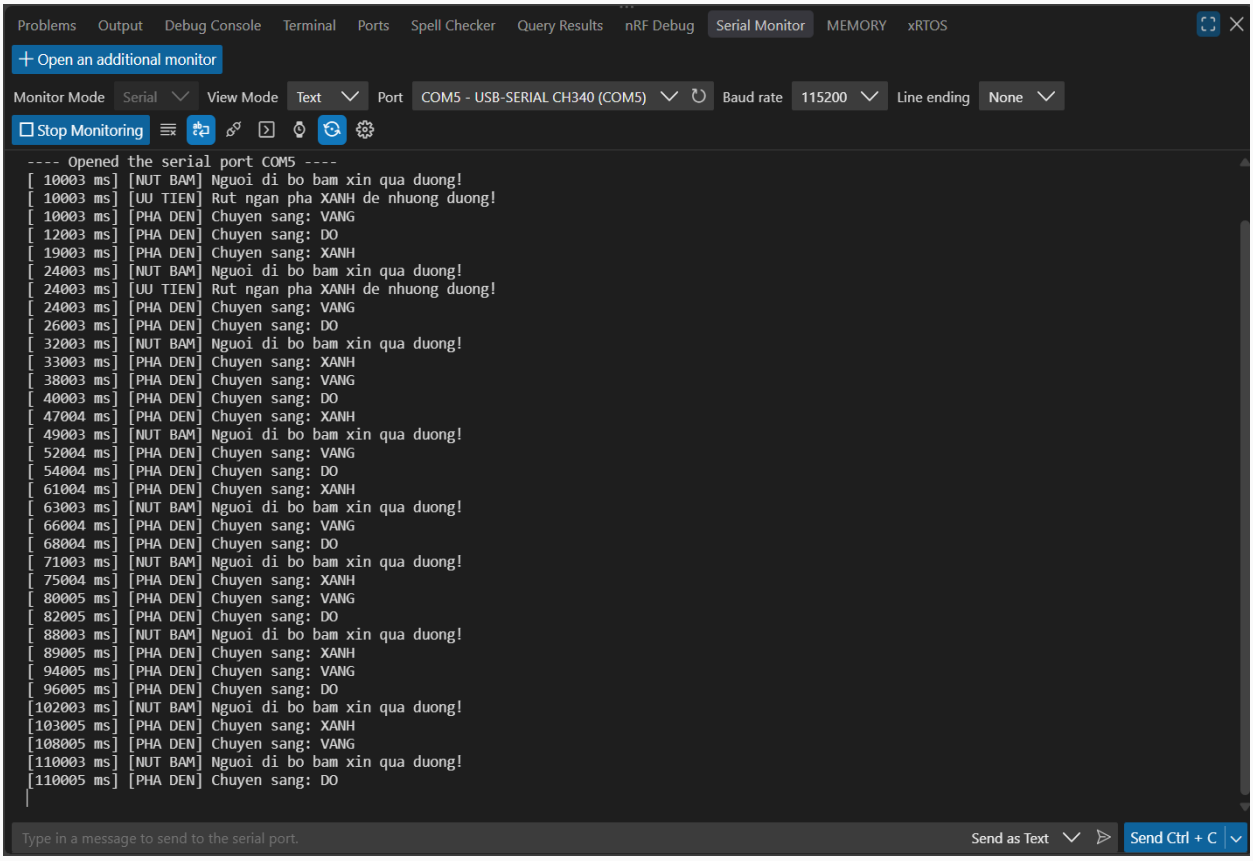
/* Thread Blinky Heartbeat */
void fn_thread_led_blinky(void *p1, void *p2, void *p3)
{
	gpio_pin_configure_dt(&led3, GPIO_OUTPUT_INACTIVE);
	while (1) {
		gpio_pin_set_dt(&led3, 1);
		k_msleep(1000);
		gpio_pin_set_dt(&led3, 0);
```

```
                k_msleep(1000);
        }
}

K_THREAD_DEFINE(thread_traffic_light, 1024, fn_thread_traffic_light, NULL, NULL, NULL, 5, 0, 0);
K_THREAD_DEFINE(thread_led_blinky, 1024, fn_thread_led_blinky, NULL, NULL, NULL, 7, 0, 0);
K_THREAD_DEFINE(thread_logger, 1024, fn_thread_logger, NULL, NULL, NULL, 10, 0, 0);

int main(void)
{
        k_timer_start(&ped_timer, K_MSEC(10000), K_NO_WAIT);
        return 0;
}
```

Bài 5 – Log Serial Monitor



Bài 5 — Trả lời câu hỏi

(a) Vì sao cần k_sem_reset trước khi chờ yêu cầu trong pha XANH? Cho ví dụ cụ thể từ log nếu bỏ lệnh này:

- Vì sao cần k_sem_reset:

+ Đèn giao thông chỉ ưu tiên cho người đi bộ khi yêu cầu được bấm trong lúc đèn xe đang XANH (sau 2 giây an toàn bắt buộc ban đầu). Nếu người đi bộ bấm nút lúc xe đang đèn Đỏ hoặc trong 2 giây đầu của đèn Xanh, yêu cầu đó là không hợp lệ. Hàm k_sem_reset(&ped_sem) đưa semaphore về 0 để xóa sạch các lần bấm cũ không hợp lệ trước đó, bảo đảm luồng chỉ phản ứng với yêu cầu mới phát sinh trong 3 giây tiếp theo.

- Ví dụ cụ thể từ log nếu bỏ lệnh này:

+ Tại mốc 32000 ms (32s), nút bấm được kích hoạt khi hệ thống đang ở PHA ĐỎ (từ 26s đến 33s).

+ Đến 33000 ms đèn chuyển sang PHA XANH và chạy 2 giây an toàn bắt buộc (đến 35s).

+ Nếu BỎ `k_sem_reset`: Đúng tại 35000 ms, lệnh `k_sem_take` sẽ lấy ngay tín hiệu cũ từ lúc 32s và rút ngắn pha xanh ngay lập tức -> Đèn xanh chỉ sáng được đúng 2 giây đã chuyển sang Vàng, xe cộ vừa xuất phát đã bị ép dừng đột ngột gây tai nạn nguy hiểm.

+ Có `k_sem_reset`: Tín hiệu lúc 32s bị xóa, đèn XANH sáng trọn vẹn 5 giây (đến 38s) trước khi chuyển sang Vàng.

(b) Vì sao dùng một thread logger duy nhất thay vì để mọi thread tự printf? Vì sao logger nên có ưu tiên thấp nhất?

- Vì sao dùng một thread logger duy nhất:

+ Tránh xung đột ngoại vi UART: Cổng Serial UART là tài nguyên chia sẻ phần cứng. Nếu nhiều thread cùng gọi `printf()`, các ký tự sẽ bị xen kẽ làm hỏng dòng log in ra.

+ Bảo đảm tính thời gian thực (Real-time determinism): Hàm `printf()` qua UART rất chậm. Sử dụng Message Queue với `k_msgq_put(..., K_NO_WAIT)` giúp các thread điều khiển và ISR chỉ mất vài micro-giây để đẩy sự kiện vào hàng đợi rồi quay lại làm việc ngay, không bị trễ thời gian điều khiển.

- Vì sao logger nên có ưu tiên thấp nhất (Priority 10):

+ Việc in thông tin ra màn hình chỉ là tác vụ phụ (giám sát). Các tác vụ điều khiển đèn giao thông an toàn (Priority 5) và báo nhịp tim hệ thống (Priority 7) phải luôn được ưu tiên CPU cao hơn. Đặt logger ở mức thấp nhất bảo đảm nó chỉ chạy khi các tác vụ quan trọng đã xử lý xong hoặc đang ngủ.

(c) Mô tả dòng thời gian 0–40 s: các pha đèn, các thời điểm có yêu cầu, và những lần pha XANH bị rút ngắn:

1. 0s - 7s (0 - 7000 ms): Đèn ĐỎ (Y0 bật).
2. 7s - 10s (7000 - 10000 ms): Đèn XANH (Y1 bật).
   - Tại t = 10s (10000 ms): Có yêu cầu bấm nút lần 1 (sau 2s an toàn).
   - ***RÚT NGẮN PHA XANH LẦN 1***: Đèn xanh kết thúc ngay tại giây thứ 10 (chỉ sáng 3s thay vì 5s).
3. 10s - 12s (10000 - 12000 ms): Đèn VÀNG (Cả Y0 và Y1 cùng bật) kéo dài 2s.
4. 12s - 19s (12000 - 19000 ms): Đèn ĐỎ (Y0 bật) kéo dài 7s.
5. 19s - 24s (19000 - 24000 ms): Đèn XANH (Y1 bật). Không có ai bấm nút trong cửa sổ chờ 21s - 24s -> Pha xanh chạy trọn vẹn 5s bình thường.
6. 24s - 26s (24000 - 26000 ms): Đèn VÀNG (Cả Y0 và Y1 cùng bật).
   - Tại t = 24s (24000 ms): Có yêu cầu bấm nút lần 2 (10s + 14s = 24s) rơi trúng lúc đèn Vàng -> Không có tác dụng rút ngắn.
7. 26s - 33s (26000 - 33000 ms): Đèn ĐỎ (Y0 bật) kéo dài 7s.
   - Tại t = 32s (32000 ms): Có yêu cầu bấm nút lần 3 (24s + 8s = 32s) rơi vào đèn Đỏ -> Sẽ bị k_sem_reset xóa bỏ ở giây 35.
8. 33s - 38s (33000 - 38000 ms): Đèn XANH (Y1 bật).
   - Tại t = 35s: Gọi k_sem_reset(), yêu cầu ở giây 32 bị xóa. Không có nút bấm mới trong 3s tiếp theo -> Pha xanh chạy trọn vẹn 5s.
9. 38s - 40s (38000 - 40000 ms): Đèn VÀNG (Cả Y0 và Y1 cùng bật).

* ***Kết luận:*** Trong khoảng 0 - 40 giây, chỉ có DUY NHẤT 1 LẦN pha XANH bị rút ngắn tại mốc 10000 ms (giây thứ 10).